

Atividade Prática - P1: Sistema de Gestão de Biblioteca (Sisbiblioteca)

Este tutorial orienta o desenvolvimento passo a passo de uma aplicação backend em **Java 17+** com **Spring Boot**, **Spring Data JPA**, banco de dados em memória **H2** e interface de linha de comando via **CommandLineRunner**, baseada na arquitetura definida para o projeto **sisbiblioteca** ([Material Didático - Módulo 03](#)).

1. Visão Geral e Objetivos

- **Domínio:** Cadastro e consulta de **Autores** e seus respectivos **Livros**.
 - **Conceito-chave:** Mapeamento Objeto-Relacional (ORM) com relacionamento **@ManyToOne** e **@OneToMany**.
 - **Interface:** Menu interativo no console utilizando **CommandLineRunner** e **Scanner**, além do console Web do H2.
-

2. Estrutura do Projeto

```
sisbiblioteca/  
├── pom.xml  
└── src/  
    ├── main/  
    │   ├── java/com/lab/jpa/sisbiblioteca/  
    │   │   ├── SisbibliotecaApplication.java  
    │   │   ├── config/  
    │   │   │   └── DataInitializer.java  
    │   │   ├── model/  
    │   │   │   ├── Autor.java  
    │   │   │   └── Livro.java  
    │   │   └── repository/  
    │   │       ├── AutorRepository.java  
    │   │       └── LivroRepository.java  
    └── resources/  
        └── application.properties
```

3. Passo 1: Criação do Projeto (Spring Initializr)

No VS Code (**Ctrl + Shift + P** > **Spring Initializr: Create a Maven Project**) ou via start.spring.io:

- **Project:** Maven
- **Language:** Java
- **Spring Boot:** 3.x ou 4.x

- **Group Id:** `com.lab.jpa`
- **Artifact Id:** `sisbiblioteca`
- **Name:** `sisbiblioteca`
- **Package Name:** `com.lab.jpa.sisbiblioteca`
- **Packaging Type:** Jar
- **Java Version:** 17 ou superior
- **Dependencies:**

1. `Spring Data JPA`
 2. `H2 Database`
 3. `Spring Web`
 4. `Lombok`
-

4. Passo 2: Configuração do Banco de Dados (`application.properties`)

No arquivo `src/main/resources/application.properties`, configure o banco H2 em memória, a exibição das instruções SQL formatadas e o console web:

```
# Configuração do DataSource H2
spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect

# Exibição e formatação de SQL no Console
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

# Habilitação do Console Web H2
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console
```

5. Passo 3: Modelo de Domínio (Entidades JPA)

`Autor.java`

Crie a classe no pacote `com.lab.jpa.sisbiblioteca.model`:

```
package com.lab.jpa.sisbiblioteca.model;

import jakarta.persistence.*;
import lombok.AllArgsConstructor;
import lombok.Data;
```

```

import lombok.NoArgsConstructor;
import lombok.ToString;

import java.util.ArrayList;
import java.util.List;

@Entity
@Table(name = "autores")
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Autor {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 100)
    private String nome;

    @OneToMany(mappedBy = "autor", cascade = CascadeType.ALL, fetch =
FetchType.LAZY)
    @ToString.Exclude
    private List<Livro> livros = new ArrayList<>();

    public Autor(String nome) {
        this.nome = nome;
    }
}

```

Livro.java

Crie a classe no pacote `com.lab.jpa.sisbiblioteca.model`:

```

package com.lab.jpa.sisbiblioteca.model;

import jakarta.persistence.*;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Entity
@Table(name = "livros")
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Livro {

    @Id

```

```

    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 150)
    private String titulo;

    @Column(name = "ano_publicacao")
    private Integer anoPublicacao;

    @ManyToOne(fetch = FetchType.EAGER)
    @JoinColumn(name = "autor_id", nullable = false)
    private Autor autor;

    public Livro(String titulo, Integer anoPublicacao, Autor autor) {
        this.titulo = titulo;
        this.anoPublicacao = anoPublicacao;
        this.autor = autor;
    }
}

```

6. Passo 4: Camada de Persistência (Repositories)

AutorRepository.java

Crie no pacote `com.lab.jpa.sisbiblioteca.repository`:

```

package com.lab.jpa.sisbiblioteca.repository;

import com.lab.jpa.sisbiblioteca.model.Autor;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.util.List;

@Repository
public interface AutorRepository extends JpaRepository<Autor, Long> {
    List<Autor> findByNomeContainingIgnoreCase(String nome);
}

```

LivroRepository.java

Crie no pacote `com.lab.jpa.sisbiblioteca.repository`:

```

package com.lab.jpa.sisbiblioteca.repository;

```

```
import com.lab.jpa.sisbiblioteca.model.Livro;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.util.List;

@Repository
public interface LivroRepository extends JpaRepository<Livro, Long> {
    List<Livro> findByTituloContainingIgnoreCase(String titulo);
    List<Livro> findById(Long autorId);
}
```

7. Passo 5: Menu Interativo (CommandLineRunner)

Crie a classe `DataInitializer.java` no pacote `com.lab.jpa.sisbiblioteca.config`. Esta classe executa imediatamente após a inicialização do contexto Spring e fornece o menu interativo via terminal.

```
package com.lab.jpa.sisbiblioteca.config;

import com.lab.jpa.sisbiblioteca.model.Autor;
import com.lab.jpa.sisbiblioteca.model.Livro;
import com.lab.jpa.sisbiblioteca.repository.AutorRepository;
import com.lab.jpa.sisbiblioteca.repository.LivroRepository;
import org.springframework.boot.CommandLineRunner;
import org.springframework.stereotype.Component;

import java.util.Optional;
import java.util.Scanner;

@Component
public class DataInitializer implements CommandLineRunner {

    private final AutorRepository autorRepository;
    private final LivroRepository livroRepository;

    public DataInitializer(AutorRepository autorRepository, LivroRepository
livroRepository) {
        this.autorRepository = autorRepository;
        this.livroRepository = livroRepository;
    }

    @Override
    public void run(String... args) throws Exception {
        var scanner = new Scanner(System.in);
        var continuar = true;

        System.out.println("=====");
        System.out.println("    SISTEMA DE GESTÃO DE BIBLIOTECA JPA    ");
        System.out.println("=====");
    }
}
```

```

while (continuar) {
    System.out.println("\nMENU DE OPÇÕES:");
    System.out.println("1 - Cadastrar Autor");
    System.out.println("2 - Listar Autores");
    System.out.println("3 - Cadastrar Livro");
    System.out.println("4 - Listar Livros");
    System.out.println("0 - Sair");
    System.out.print("Escolha uma opção: ");

    var opcao = scanner.nextLine();

    continuar = switch (opcao) {
        case "1" -> {
            cadastrarAutor(scanner);
            yield true;
        }
        case "2" -> {
            listarAutores();
            yield true;
        }
        case "3" -> {
            cadastrarLivro(scanner);
            yield true;
        }
        case "4" -> {
            listarLivros();
            yield true;
        }
        case "0" -> {
            System.out.println("Encerrando aplicação...");
            yield false;
        }
        default -> {
            System.out.println("Opção inválida! Tente novamente.");
            yield true;
        }
    };
}

System.out.println("Aplicação finalizada.");
}

private void cadastrarAutor(Scanner scanner) {
    System.out.print("Digite o nome do autor: ");
    var nome = scanner.nextLine();

    if (nome.isBlank()) {
        System.out.println("Nome inválido!");
        return;
    }

    var autor = new Autor(nome);
    autorRepository.save(autor);
}

```

```

        System.out.println(">>> Autor '" + autor.getNome() + "' cadastrado com ID: " + autor.getId());
    }

    private void listarAutores() {
        var autores = autorRepository.findAll();
        if (autores.isEmpty()) {
            System.out.println("Nenhum autor cadastrado.");
            return;
        }

        System.out.println("\n--- LISTA DE AUTORES ---");
        autores.forEach(a -> System.out.printf("ID: %d | Nome: %s\n", a.getId(), a.getNome()));
        System.out.println("-----");
    }

    private void cadastrarLivro(Scanner scanner) {
        listarAutores();
        System.out.print("Informe o ID do autor do livro: ");
        var idStr = scanner.nextLine();

        try {
            var autorId = Long.parseLong(idStr);
            Optional<Autor> autorOpt = autorRepository.findById(autorId);

            if (autorOpt.isEmpty()) {
                System.out.println("Autor não encontrado com o ID informado!");
                return;
            }

            System.out.print("Digite o título do livro: ");
            var titulo = scanner.nextLine();

            System.out.print("Digite o ano de publicação: ");
            var ano = Integer.parseInt(scanner.nextLine());

            var livro = new Livro(titulo, ano, autorOpt.get());
            livroRepository.save(livro);
            System.out.println(">>> Livro '" + livro.getTitulo() + "' cadastrado com sucesso!");
        } catch (NumberFormatException e) {
            System.out.println("Valor numérico inválido informado.");
        }
    }

    private void listarLivros() {
        var livros = livroRepository.findAll();
        if (livros.isEmpty()) {
            System.out.println("Nenhum livro cadastrado.");
            return;
        }
    }

```

```

        System.out.println("\n--- LISTA DE LIVROS ---");
        livros.forEach(l -> System.out.printf("ID: %d | Título: %s | Ano: %d |
Autor: %s\n",
            l.getId(), l.getTitulo(), l.getAnoPublicacao(),
            l.getAutor().getNome()));
        System.out.println("-----");
    }
}

```

8. Passo 6: Classe Principal da Aplicação

Verifique a classe `SisbibliotecaApplication.java` no pacote raiz `com.lab.jpa.sisbiblioteca`:

```

package com.lab.jpa.sisbiblioteca;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class SisbibliotecaApplication {

    public static void main(String[] args) {
        SpringApplication.run(SisbibliotecaApplication.class, args);
    }
}

```

9. Passo 7: Execução e Validação

1. Execução no Terminal:

- Pelo VS Code ou terminal: `mvn spring-boot:run` ou execute o `main` da classe `SisbibliotecaApplication`.
- O menu interativo será exibido no console.

2. Fluxo de Teste Sugerido:

- Escolha opção **1** para cadastrar autores (ex: *Machado de Assis*, *Clarice Lispector*).
- Escolha opção **3** para cadastrar livros associados aos IDs dos autores cadastrados (ex: *Dom Casmurro*, *A Hora da Estrela*).
- Escolha opções **2** e **4** para listar e validar a persistência e associação.

3. Validação via H2 Console:

- Abra no navegador: `http://localhost:8080/h2-console`
- **JDBC URL:** `jdbc:h2:mem:testdb`

- **User Name:** **sa**
- **Password:** *(deixar em branco)*
- Clique em **Connect** para visualizar as tabelas **AUTORES** e **LIVROS** com suas respectivas chaves estrangeiras.